

# 05

## CHANGELOOP

### Question 5

What changes at cluster scale, and which parts are not solved yet.

---

#### The question

---

**What changes at scale?**

#### My answer

---

Scaling this is not a matter of copying the current model four hundred times. The engineering problem genuinely changes, and I would rather set out what breaks than claim it simply grows.

Measurement has to become real first. Nothing in the system is measured today. Two sensors change that: evaporator steam flow, and reject conductivity. Those two readings turn the two coefficients carrying the most weight from published values into measured ones. They also need calibration and health monitoring, because a drifting sensor is worse than no sensor at all. The rule already built in is that a reading which is missing, frozen or implausible makes the system hold and credit zero rather than guess. That rule has to survive scale instead of being relaxed for throughput.

Then there is variability. Salt concentration shifts with shade depth and dye chemistry. Membrane behaviour changes as membranes age and foul. Right now I model the concentration ceiling as one number, 60,000 mg/L. A pilot has to establish whether that holds across a real operating range, or whether it needs to become a function of membrane condition.

The largest gap I know about is shared treatment capacity.

My optimiser handles one machine's queue. A real mill runs many machines into one membrane system and one evaporator, and a CETP serves hundreds of units feeding one shared plant. That is a different optimisation problem, not a bigger version of the same one. Machines compete for shared capacity, and the best plan for a single machine is not the best plan for the plant.

**I have not solved that, and I am not going to pretend it is a detail.  
It is the scale-up problem.**

There is a fourth thing that is easy to miss. The scheduling decision happens daily, but the decision that moves the energy bill most, switching to low-salt chemistry, is a procurement and quality decision running on a much slower cycle. At scale the system has to serve both clocks: a daily recommendation for the planner, and a periodic case for whoever buys chemicals and signs off recipes.

Operationally the planner uses it while scheduling, the quality supervisor keeps authority over wash-off release, the unit owner takes the financial benefit, and the CETP is the most efficient commercial channel, because one connection can serve many member units and its incentive already points the same way. Less salt arriving means less steam bought and less salt to store.

My rollout runs about 32 weeks. Baseline metering, then shadow mode where recommendations are produced but change nothing, then advisory use with human approval, then a controlled wash-off release trial, then the chemistry decision, then expansion to more units. A quality failure, particularly a wash-fastness failure, stops that stage rather than being absorbed to protect a target.

The point of that sequence is not to prove I was right. It is to find out whether the decision logic survives real variability, shared equipment and real constraints. If it does, the same architecture extends from one machine toward plant and cluster level. If it does not, better to know in week six than after a cluster has paid for it.

## Supporting notes

### Why the answer is shaped this way

Scale questions usually get answered with ambition. This one gets answered with the three specific things that break, because naming your own largest gap is more convincing than implying you do not have one. The shared-treatment gap in particular is real, and a process engineer on the panel will spot it immediately if it is hidden.

The staged rollout is there to show an understanding of adoption risk alongside technical risk. Shadow mode matters most: it is how you earn the right to influence a factory's decisions, by being correct for four weeks while changing nothing.

### Claims and where they rest

| Claim                          | Basis                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Nothing currently measured     | <code>core/factors.py</code> : 0 coefficients classed MEASURED, enforced by a test                                  |
| Two readings unlock the most   | Sensitivity sweep: steam cost and reject ceiling dominate the result                                                |
| Fail-closed on bad sensor data | <code>core/telemetry.py</code> ; <code>automatic_release</code> hardcoded <code>False</code> ; 5 fault modes tested |
| Missing data credits zero      | Safety-interlock tests in <code>tests/test_engine.py</code>                                                         |
| Ceiling is one modelled figure | <code>ro_max_reject_tds_mg_l</code> = 60,000 mg/L, inside a published 15,000–80,000 band                            |
| Optimiser handles one machine  | <code>core/optimizer.py</code> ; exhaustive to 8 lots, heuristic beyond that                                        |
| CETP incentive aligns          | Less salt means less steam and less stored salt; over 1 lakh tonnes currently held with no disposal route           |
| 32-week staged rollout         | <code>/api/pilot</code> ; the phases are defined in the engine, not written for this answer                         |

### What the answer deliberately avoids claiming

- No claim that the shared-treatment optimisation is solved or nearly solved.
- No claim that the 60,000 mg/L ceiling holds universally.
- No cluster rollout timeline. The 32 weeks covers one unit through to first expansion, not a cluster.

### The hardest follow-up

**"You've solved the easy version. The real problem is the shared plant."**

That is correct, which is why it is in the answer rather than waiting to be raised. What exists is the decision layer and the physics driving it, proven on a single machine, with the accounting and the safety behaviour already working. The shared-resource optimisation sits on top of the same consequence model, and the thing it needs to know, that reject volume is set by salt, does not change when the plant is shared. That is why I started here rather than starting with the harder scheduling problem on top of a cost model nobody had examined.